MỤC LỤC

Chương 1. Giới thiệu chung 1
1.1. Mục tiêu dự án 1
1.2. Phạm vi dự án 1
1.3. Ngoài phạm vi dự án 2
Chương 2. Kiến trúc tổng quan 2
2.1. Phần cứng 2
2.2. Tầng mạng 3
2.3. Tầng ứng dụng 4
2.4. Công nghệ sử dụng 4
Chương 3. Thiết kế chi tiết hệ thống 5
3.1. Thiết kế cơ sở dữ liệu 5
3.2. Các Use Case 7
3.2.1. Use Case 1 - Xem dữ liệu dashboard 8
3.2.2. Use Case 2 - Điều khiển thiết bị 10
3.2.3. Use Case 3 - Xem lịch sử dữ liệu cảm biến 13
3.2.4. Use Case 4 - Xem lịch sử hoạt động thiết bị 16
3.2.5. Use Case 5 - Xem thông tin profile 19
Chương 4 Kết luận 22

Chương 1. Giới thiệu chung

1.1. Mục tiêu dự án

Xây dựng một hệ thống IoT sử dụng ESP32 để thu thập dữ liệu từ các cảm biến môi trường và điều khiển các thiết bị LED thông qua giao diện Web.

Hệ thống có các chức năng chính:
- Thu thập dữ liệu nhiệt độ, độ ẩm và cường độ ánh sáng từ các cảm biến.
- Cập nhật dữ liệu cảm biến lên giao diện Web định kỳ mỗi 2 giây.
- Cho phép người dùng điều khiển bật/tắt các đèn LED thông qua giao diện Web.
- Lưu trữ lịch sử dữ liệu cảm biến để người dùng có thể xem lại.
- Lưu trữ lịch sử bật/tắt thiết bị để người dùng có thể theo dõi các thao tác đã thực hiện.

1.2. Phạm vi dự án

Hệ thống được xây dựng dưới dạng mô hình IoT trong phòng, sử dụng ESP32 làm bộ điều khiển trung tâm.
Hệ thống bao gồm:

- Thu thập dữ liệu nhiệt độ và độ ẩm thông qua cảm biến DHT11.
- Thu thập dữ liệu cường độ ánh sáng thông qua cảm biến ánh sáng.
- Điều khiển bật/tắt các đèn LED thông qua giao diện Web.
- Cập nhật dữ liệu cảm biến lên Web định kỳ mỗi 2 giây.
- Lưu trữ dữ liệu cảm biến vào cơ sở dữ liệu.
- Lưu trữ lịch sử bật/tắt các thiết bị.
- Cho phép người dùng xem dữ liệu cảm biến và lịch sử hoạt động thông qua Web.

1.3. Ngoài phạm vi dự án

Các chức năng sau không thuộc phạm vi của hệ thống:
- Ứng dụng dành riêng cho điện thoại di động.
- Quản lý nhiều tài khoản người dùng.
- Phân quyền người dùng.
- Phân tích hoặc dự đoán dữ liệu bằng AI.
- Điều khiển các thiết bị điện thực tế ngoài mô hình LED.

Chương 2. Kiến trúc tổng quan

2.1. Phần cứng

- Khối điều khiển trung tâm: Sử dụng vi điều khiển ESP32 đóng vai trò thu thập dữ liệu và gửi phản hồi xác nhận trạng thái thực thi về Server.
- Cảm biến: Bao gồm cảm biến nhiệt độ, độ ẩm và cường độ ánh sáng.
- Thiết bị thực thi: Hệ thống bóng LED dùng để mô phỏng cho các thiết bị điều khiển thực tế.
- Kết nối: Sử dụng Wi-Fi để giao tiếp với hệ thống thông qua giao thức MQTT, hỗ trợ truyền lệnh điều khiển và phản hồi trạng thái thiết bị.

Mạch bao gồm các thành phần:

Thành phần	Hình ảnh thành phần
Mạch điều khiển ESP32	
Cảm biến nhiệt độ, độ ẩm DHT11	
Cảm biến ánh sáng 4 pin	

Led 5mm	
Breadboard	
Dây nối	

2.2. Tầng mạng

Lớp này đóng vai trò trung gian điều phối luồng dữ liệu:

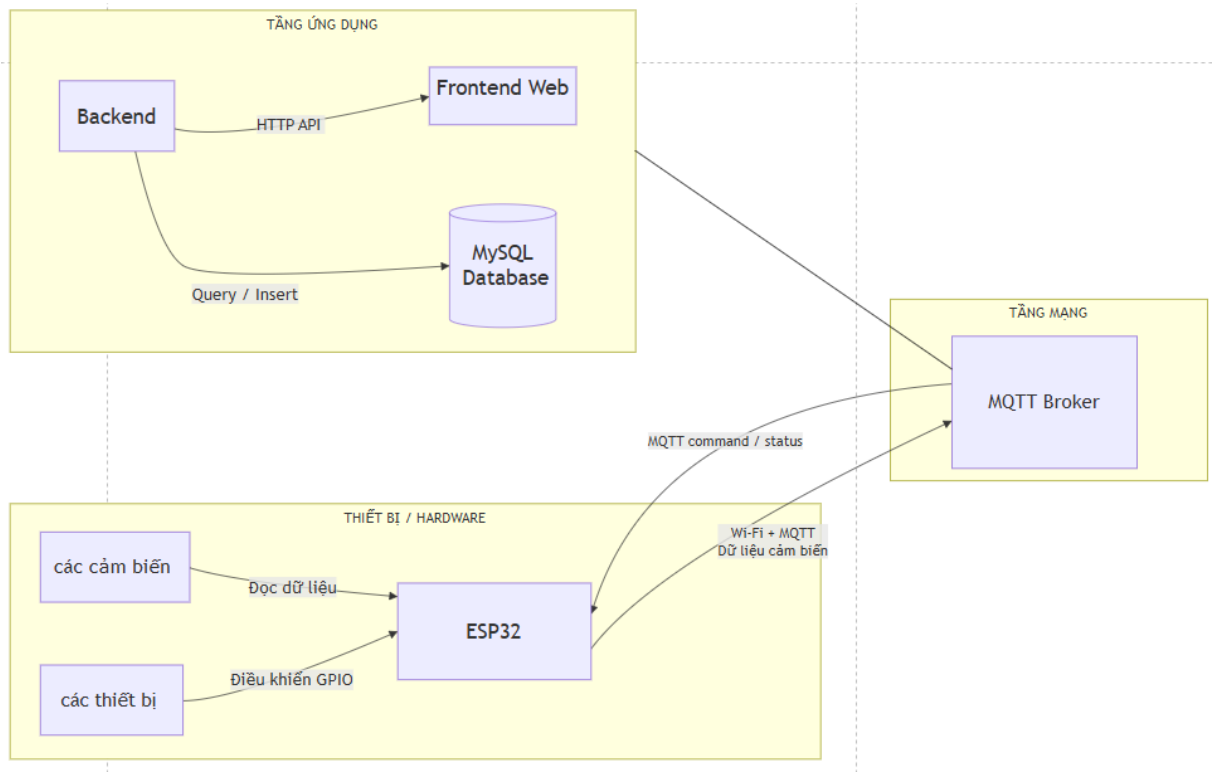
MQTT Broker: Trạm trung chuyển bản tin. Phần cứng và Server giao tiếp qua các Topic quy định sẵn như iot/sensors/data (để gửi dữ liệu môi trường), iot/devices/command (để nhận lệnh) và iot/devices/status (để xác nhận thực thi).

2.3. Tầng ứng dụng

Đây là nơi xử lý logic nghiệp vụ và cung cấp giao diện quản trị:

- Backend Server: Tiếp nhận dữ liệu từ MQTT, thực hiện logic điều khiển 2 pha. Server chỉ cập nhật trạng thái lệnh thành công (SUCCESS) sau khi nhận được thông điệp xác nhận từ mạch, đảm bảo tính nhất quán giữa thực tế và phần mềm. Giao tiếp giữa Frontend Web và Backend Server sử dụng giao thức HTTP API theo cơ chế định kỳ (Polling mỗi 2 giây) để cập nhật dữ liệu cảm biến và đồng bộ trạng thái hệ thống.
- Database (MySQL): Lưu trữ thông tin dữ liệu đo lường của cảm biến và nhật ký hoạt động của thiết bị
- Frontend: Cung cấp giao diện tương tác với các trang bao gồm: Dashboard (cập nhật định kỳ mỗi 2 giây của cảm biến và điều khiển thiết bị), Lịch sử cảm biến, Lịch sử bật/tắt thiết bị và Profile.

Sơ đồ:



2.4. Công nghệ sử dụng

- Phần cứng: Mạch ESP32, Cảm biến DHT11, Cảm biến ánh sáng, 3 đèn LED, Breadboard, dây nối
- Phần mềm & Môi trường:
  1) Môi trường code phần cứng: Visual Studio Code sử dụng extension PlatformIO IDE.
  2) MQTT Broker: Sử dụng MQTT Broker tiêu chuẩn (hỗ trợ triển khai linh hoạt qua Eclipse Mosquitto cục bộ hoặc dịch vụ điện toán đám

mây như HiveMQ Cloud) để định tuyến các bản tin giữa ESP32 và Server.

3) Backend: Node.js (Express framework), thư viện mysql2, mqtt.
4) Frontend: HTML, CSS, JavaScript, thư viện Chart.js vẽ biểu đồ.

Chương 3. Thiết kế chi tiết hệ thống

3.1. Thiết kế cơ sở dữ liệu

Sensor (Bảng danh mục cảm biến)

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	Primary Key, Auto Increment	Khóa chính
name	VARCHAR(100)	Not Null	Tên cảm biến

```
-- Bảng Sensor
CREATE TABLE sensor (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL
);
```

data_sensor (Bảng dữ liệu cảm biến)

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	Primary Key, Auto Increment	Khóa chính
sensor_id	INT	Foreign Key, Not Null	Nối với cột id của bảng sensors
value	FLOAT	Not Null	Giá trị đo được từ cảm biến
time	TIMESTAMP	Default Current_Timestamp	Thời gian đo

```
-- Bảng data_sensor
CREATE TABLE data_sensor (
   id INT AUTO_INCREMENT PRIMARY KEY,
   sensor_id INT NOT NULL,
   value FLOAT NOT NULL,
   time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (sensor_id) REFERENCES Sensor(id)
);
```

Devices (Bảng danh mục thiết bị)

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả

id	INT	Primary Key, Auto Increment	Khóa chính
name	VARCHAR(100)	Not Null	Tên thiết bị

```
-- Bảng devices
CREATE TABLE Devices (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);
```

active_history (Bảng lịch sử điều khiển thiết bị)

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
id	INT	Primary Key, Auto Increment	Số thứ tự bản ghi
device_id	INT	Foreign Key, Not Null	Nối với cột id của bảng devices
action	VARCHAR(20)	Not Null	Lệnh thao tác
status	VARCHAR(20)	Not Null	Trạng thái thực tế của thiết bị sau khi thực hiện lệnh
time	TIMESTAMP	Default Current_Timestamp	Thời gian thực hiện
profile_id	INT	Foreign Key, Not Null	Nối với cột id của bảng profile

```
-- Bảng active_history
CREATE TABLE active_history (
    id INT AUTO_INCREMENT PRIMARY KEY,
    device_id INT NOT NULL,
    profile_id INT NOT NULL,
    action VARCHAR(20) NOT NULL,
    status VARCHAR(20) NOT NULL,
    time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (device_id) REFERENCES Devices(id),
    FOREIGN KEY (profile_id) REFERENCES Profile(id)
);
```

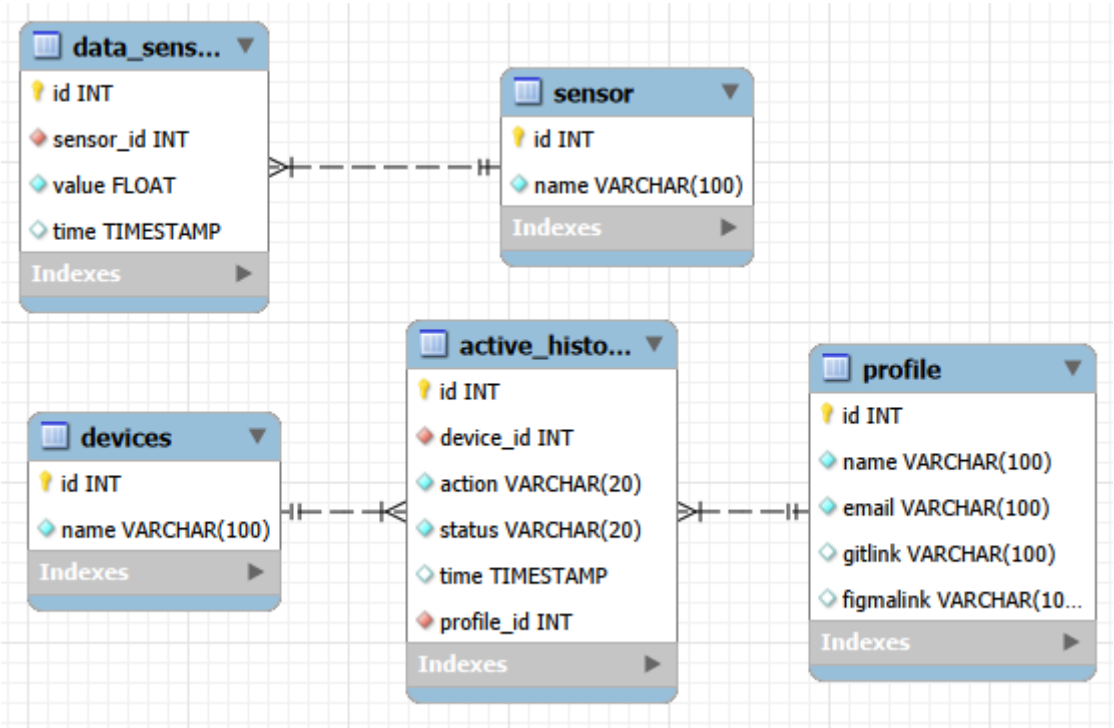
Profile (Bảng thông tin hồ sơ)

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả

id	INT	Primary Key, Auto Increment	Khóa chính
name	VARCHAR(100)	Not Null	Họ và tên
email	VARCHAR(100)	Not Null	Email liên hệ
gitlink	VARCHAR(100)	Not Null	Link Github
figmalink	VARCHAR(100)	Not Null	Link Figma

-- Bảng Profile

```
CREATE TABLE profile (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    gitlink VARCHAR(100) NOT NULL,
    figmalink VARCHAR(100) NOT NULL
);
```



3.2. Các Use Case

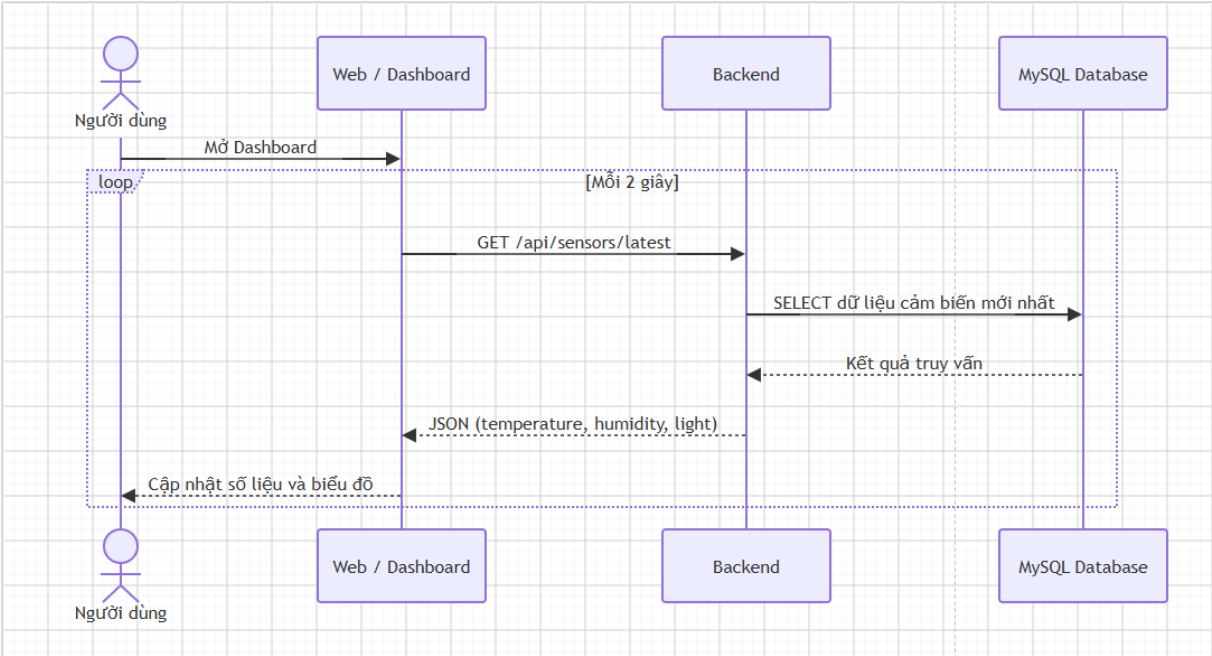
Actor (Người dùng) tương tác với hệ thống qua 5 tính năng chính: Xem Dashboard, Điều khiển thiết bị, Xem Data Sensor, Xem Active History và Xem Profile.

3.2.1. Use Case 1 - Xem dữ liệu dashboard

a. Bảng Use case

Thuộc tính	Chi tiết
Tên Use case	Giám sát dữ liệu trên Dashboard
Mô tả	Giao diện hiển thị trực quan các thông số thu thập từ cảm biến nhiệt độ, độ ẩm và ánh sáng; dữ liệu mới nhất được cập nhật định kỳ mỗi 2 giây và dữ liệu lịch sử gần nhất được lấy để vẽ biểu đồ thay đổi.
Tiền điều kiện	Hệ thống đã có kết nối mạng, người dùng truy cập vào trang Dashboard.
Điều kiện thành công	Dashboard hiển thị chính xác dữ liệu cảm biến mới nhất và biểu đồ tương ứng.
Các bước thực hiện	1. Người dùng truy cập Dashboard. 2. Frontend gửi yêu cầu GET /api/sensors/latest đến Backend. 3. Backend truy vấn dữ liệu cảm biến mới nhất và các dữ liệu gần nhất trong bảng data_sensor để phục vụ hiển thị Dashboard. 4. Database trả về dữ liệu cho Backend. 5. Backend đóng gói dữ liệu thành JSON, bao gồm: - Giá trị cảm biến hiện tại. - Danh sách dữ liệu theo thời gian để vẽ biểu đồ. 6. Frontend nhận dữ liệu và: - Hiển thị giá trị nhiệt độ, độ ẩm, ánh sáng hiện tại. - Cập nhật biểu đồ theo dữ liệu nhận được. 7. Dashboard được cập nhật định kỳ mỗi 2 giây.

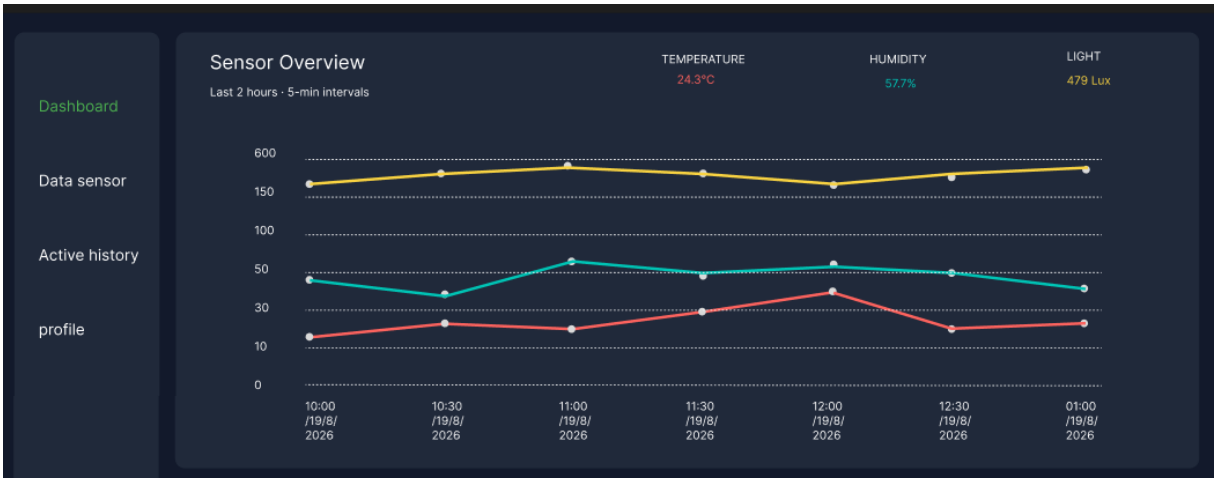
b. Biểu đồ tuần tự



Chi tiết luồng tương tác:

- **Bước 1:** Người dùng mở giao diện Dashboard trên trình duyệt.
- **Bước 2 & 3:** Định kỳ mỗi 2 giây (khung lặp loop), Web (Frontend) gửi request GET /api/sensors/latest tới Server (Backend). Backend thực hiện truy vấn các dữ liệu cảm biến gần nhất từ bảng data_sensor trong Cơ sở dữ liệu (MySQL), bao gồm cả dữ liệu mới nhất.
- **Bước 4 & 5:** Cơ sở dữ liệu trả về danh sách các bản ghi dữ liệu đo đạc gần nhất từ bảng data_sensor. Backend thực hiện tổng hợp và gom nhóm các bản ghi nhiệt độ, độ ẩm, ánh sáng có cùng mốc thời gian thành từng đối tượng dữ liệu hoàn chỉnh, sau đó đóng gói thành chuỗi JSON (gồm thời gian cùng 3 thông số temperature, humidity, light) và phản hồi về cho Web.
- **Bước 6:** Web (Frontend) tiếp nhận dữ liệu, lấy bản ghi mới nhất để cập nhật các chỉ số nhiệt độ, độ ẩm, ánh sáng hiện tại, đồng thời sử dụng toàn bộ danh sách dữ liệu nhận được để vẽ/cập nhật biểu đồ đường.

c. Giao diện



d. Mô tả API

- **Chức năng:** Trích xuất các dữ liệu đo gần nhất của nhiệt độ, độ ẩm và ánh sáng để cập nhật các giá trị hiện tại và biểu đồ trên Dashboard.
- **Method (Phương thức):** GET
- **Endpoint (Đường dẫn):** /api/sensors/latest
- **Output trả về (Thành công - Status 200 OK):**


```
{
  "status": "success",
  "data": [
    {
      "time": "17:50:04",
      "temperature": 24.1,
      "humidity": 58.2,
      "light": 475
    },
    {
      "time": "17:50:06",
      "temperature": 24.2,
      "humidity": 57.9,
      "light": 477
    },
    {
      "time": "17:50:08",
      "temperature": 24.3,
      "humidity": 57.7,
      "light": 479
    }
  ]
}
```


- **Output trả về khi lỗi:**

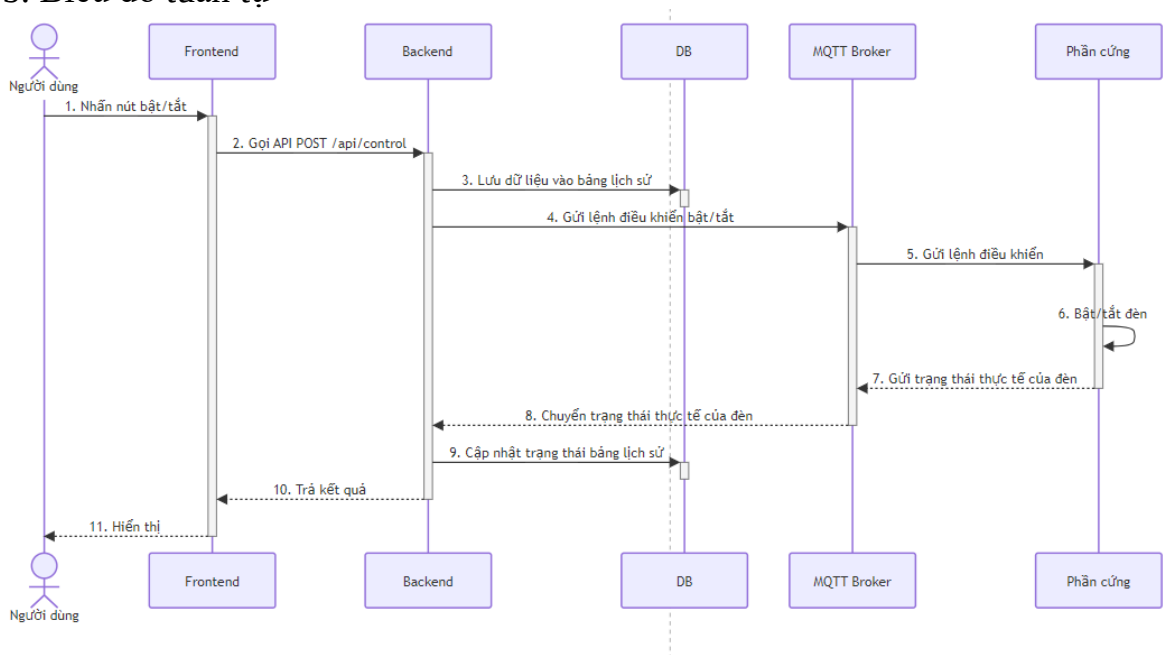

```
{
  "status": "error",
  "message": "Lỗi máy chủ khi truy vấn dữ liệu cảm biến mới nhất!"
}
```


3.2.2. Use Case 2 - Điều khiển thiết bị

a. Bảng Use case

Thuộc tính	Chi tiết
Tên Use case	Điều khiển bật tắt thiết bị
Mô tả	Cho phép người dùng điều khiển bật/tắt độc lập 3 thiết bị đèn (Đèn 1, Đèn 2, Đèn 3) trực tiếp từ giao diện Dashboard thông qua Backend và MQTT.
Tiền điều kiện	Hệ thống có kết nối mạng, ESP32 đang trực tuyến và kết nối thành công tới MQTT Broker.
Điều kiện thành công	Thiết bị phần cứng thay đổi trạng thái theo đúng lệnh và bản ghi lịch sử được cập nhật thành SUCCESS hoặc FAILED.
Các bước thực hiện	1. Người dùng nhấn nút bật/tắt thiết bị trên giao diện Dashboard. 2. Frontend gọi API POST /api/control để gửi yêu cầu điều khiển đến Backend. 3. Backend tạo bản ghi lịch sử trong cơ sở dữ liệu với action = ON/OFF và status = PENDING. 4. Backend gửi lệnh điều khiển bật hoặc tắt đến MQTT Broker. 5. MQTT Broker chuyển lệnh điều khiển đến ESP32. 6. ESP32 nhận lệnh và thực hiện bật hoặc tắt đèn. 7. ESP32 gửi trạng thái thực tế của đèn đến MQTT Broker. 8. MQTT Broker chuyển trạng thái thực tế của đèn đến Backend. 9. Backend cập nhật status của bản ghi lịch sử thành SUCCESS hoặc FAILED dựa trên phản hồi từ ESP32. 10. Backend trả kết quả thực hiện về Frontend. 11. Frontend hiển thị kết quả cho người dùng.

b. Biểu đồ tuần tự



Chi tiết luồng tương tác:

- **Bước 1 & 2:** Người dùng nhấn nút gạt bật/tắt thiết bị đèn trên Giao diện (Frontend). Frontend gửi yêu cầu qua API POST /api/control tới Server (Backend) kèm thông tin device_id và hành động action.
- **Bước 3 & 4: Backend tạo bản ghi trong bảng Active_history với action = ON/OFF và status = PENDING, đồng thời publish lệnh điều khiển lên MQTT Broker qua topic điều khiển.**
- **Bước 5 & 6:** MQTT Broker chuyển tiếp lệnh điều khiển tới vi điều khiển ESP32 (Phần cứng). ESP32 nhận lệnh và điều khiển chân GPIO để kích hoạt bật/tắt đèn LED thực tế.
- **Bước 7 & 8:** ESP32 đọc trạng thái thực tế của chân đèn và gửi thông điệp phản hồi lên MQTT Broker qua topic trạng thái. MQTT Broker tiếp tục chuyển tiếp thông điệp này về Backend.
- **Bước 9 & 10: Backend nhận phản hồi từ ESP32, đối soát kết quả và cập nhật status của bản ghi trong bảng Active_history thành SUCCESS hoặc FAILED, sau đó trả kết quả JSON về Frontend.**
- **Bước 11:** Frontend nhận phản hồi, hiển thị thông báo kết quả và cập nhật trạng thái công tắc trên màn hình cho người dùng.

c. Giao diện



d. Mô tả API: Điều khiển thiết bị

API: Điều khiển bật/tắt thiết bị

- **Chức năng:** Tiếp nhận lệnh điều khiển thiết bị từ người dùng, lưu vết thao tác và phát lệnh điều khiển phần cứng qua MQTT.
- **Method (Phương thức):** POST
- **Endpoint (Đường dẫn):** /api/control
- **Request Body:**

JSON

```
{
  "device_id": 1,
  "action": "ON"
}
```

- **Output trả về (Thành công - Status 200 OK):**

JSON
```
{
  "status": "success",
  "message": "Thiết bị đã thực hiện lệnh thành công",
  "data": {
    "device_id": 1,
    "action": "ON",
    "status": "SUCCESS"
  }
}
```

- **Output trả về (Thất bại - Status 504 Gateway Timeout / Error):**

JSON
```
{
  "status": "error",
  "message": "Thiết bị không phản hồi, kết nối MQTT hoặc phần cứng bị lỗi!",
  "data": {
    "device_id": 1,
    "action": "ON",
    "status": "FAILED"
  }
}
```

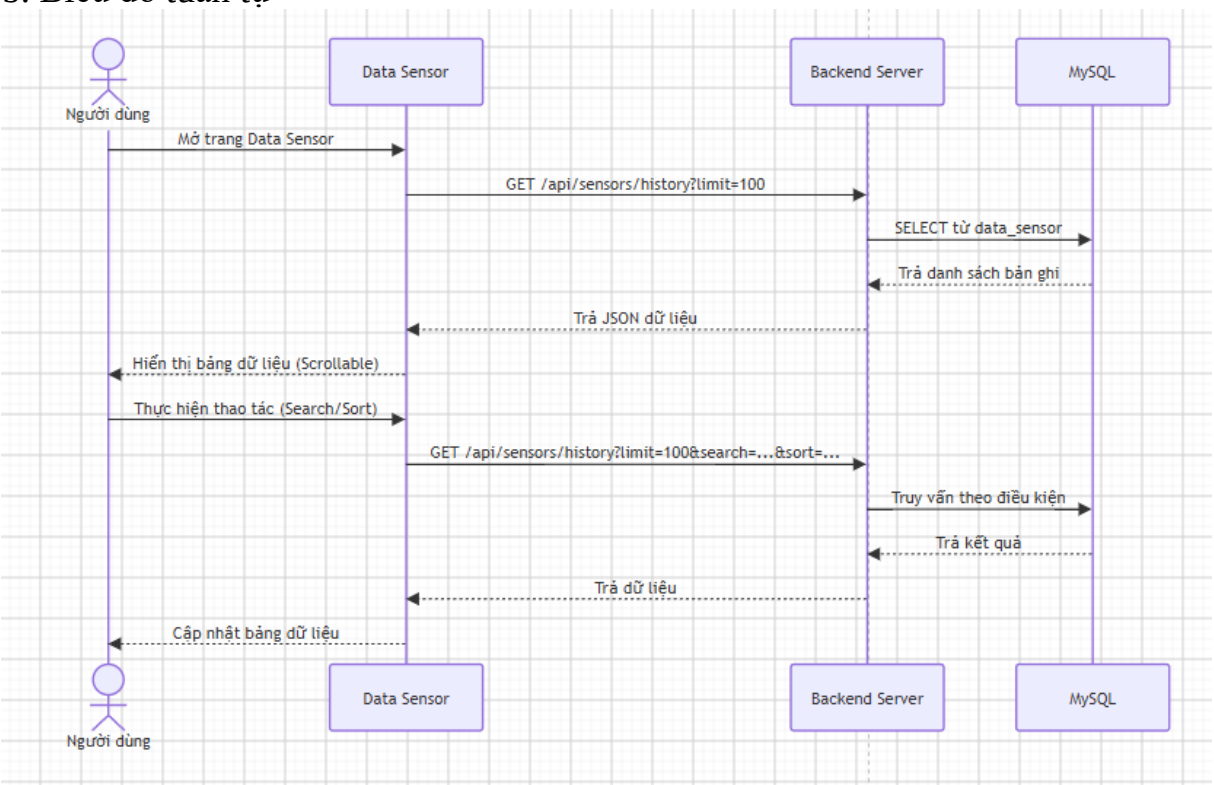
3.2.3. Use Case 3 - Xem lịch sử dữ liệu cảm biến

a. Bảng Use case

Thuộc tính	Chi tiết
Tên Use case	Xem lịch sử dữ liệu cảm biến
Mô tả	Hiển thị danh sách các bản ghi dữ liệu đo đạc của từng cảm biến (nhiệt độ, độ ẩm, ánh sáng) dưới dạng bảng cuộn dọc (scrollable) với 4 thông tin: id, tên cảm biến, giá trị và thời gian đo. Hỗ trợ các tính năng tìm kiếm (search...) và sắp xếp dữ liệu (sort...).
Tiền điều kiện	Hệ thống đã có dữ liệu đo đạc lưu trữ trong cơ sở dữ liệu.
Điều kiện thành công	Hiển thị chính xác danh sách dữ liệu khớp với điều kiện tìm kiếm hoặc sắp xếp của người dùng.
Các bước thực hiện	1. Người dùng truy cập trang Data Sensor hoặc nhập từ khóa tìm kiếm. 2. Giao diện Web gọi API GET /api/sensors/history xuống Backend. 3. Backend thực hiện truy vấn bảng data_sensor trong cơ sở dữ liệu. 4. Cơ sở dữ liệu trả danh sách bản ghi về cho Backend.

	5. Backend định dạng và trả về danh sách gồm id, sensor_name, value, time cho Web để hiển thị lên bảng.

b. Biểu đồ tuần tự



Chi tiết luồng tương tác:

- **Bước 1 & 2:** Người dùng chọn chuyển đến trang Data Sensor. Frontend tự động gửi request GET /api/sensors/history?limit=100 lên Backend để lấy dữ liệu mặc định ban đầu.
- **Bước 3 & 4:** Backend thực thi truy vấn SELECT từ bảng data_sensor trong Cơ sở dữ liệu (MySQL). Cơ sở dữ liệu trả danh sách các bản ghi dữ liệu cảm biến đã lưu về cho Backend.
- **Bước 5 & 6:** Backend đóng gói danh sách thành chuỗi JSON và trả về cho Frontend. Giao diện hiển thị danh sách các thông số đo lên bảng dữ liệu dạng cuộn (scrollable).
- **Bước 7 & 8:** Người dùng thực hiện tìm kiếm từ khóa hoặc lựa chọn sắp xếp trên thanh công cụ. Frontend gửi request bổ sung tham số GET /api/sensors/history?limit=100&search=...&sort=... lên Backend.
- **Bước 9, 10 & 11:** Backend phân tích tham số và thực thi truy vấn theo điều kiện lọc tới Cơ sở dữ liệu. Cơ sở dữ liệu trả tập kết quả phù hợp, sau đó Backend gửi phản hồi dữ liệu JSON về cho Frontend.
- **Bước 12:** Frontend cập nhật lại danh sách hiển thị trên bảng dữ liệu cho người dùng theo đúng kết quả tìm kiếm/sắp xếp.

c. Giao diện



d. Mô tả API: Tra cứu lịch sử cảm biến

API: Lấy danh sách lịch sử cảm biến

- **Chức năng:** Trích xuất danh sách lịch sử các lần đo nhiệt độ, độ ẩm và ánh sáng; hỗ trợ lọc, tìm kiếm.
- **Method (Phương thức):** GET
- **Endpoint (Đường dẫn):** /api/sensors/history
- **Query Parameters:** limit=100&search=...&sort=desc
- **Output trả về (Thành công - Status 200 OK):**

JSON


```
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "sensor_name": "Nhiệt độ",
      "value": 33,
      "time": "17:50:08, 10/07/2026"
    },
    {
      "id": 2,
      "sensor_name": "Độ ẩm",
      "value": 50,
      "time": "17:50:08, 10/07/2026"
    },
```

```
    {
      "id": 3,
      "sensor_name": "Ánh sáng",
      "value": 234,
      "time": "17:50:08, 10/07/2026"
    }
  ]
}
```


- **Output trả về (Thất bại - Status 400 Bad Request):**


```
JSON
{
  "status": "error",
  "message": "Tham số tìm kiếm hoặc sắp xếp không hợp lệ!"
}
```

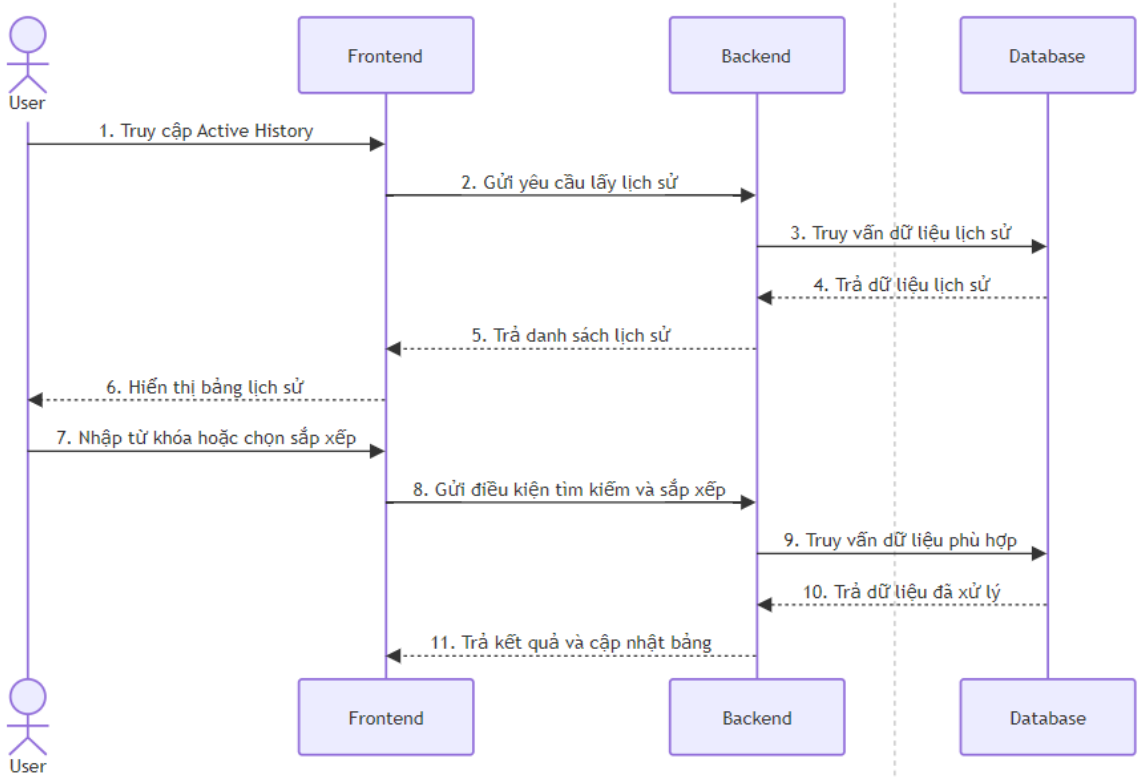

3.2.4. Use Case 4 - Xem lịch sử hoạt động thiết bị

a. Bảng Use case

Thuộc tính	Chi tiết
Tên Use case	Xem lịch sử hoạt động
Mô tả	Cho phép người dùng xem lịch sử hoạt động của các thiết bị dưới dạng bảng cuộn dọc. Mỗi bản ghi hiển thị gồm ID, tên thiết bị, action (ON/OFF), status (PENDING/ SUCCESS/FAILED) và thời gian thực hiện. Người dùng có thể tìm kiếm và lựa chọn cách sắp xếp dữ liệu.
Tiền điều kiện	Hệ thống đã có dữ liệu nhật ký điều khiển thiết bị lưu trong cơ sở dữ liệu.
Điều kiện thành công	Bảng hiển thị chính xác danh sách lịch sử hoạt động thiết bị, gồm ID, tên thiết bị, action (ON/OFF), status (PENDING/ SUCCESS/FAILED) và thời gian thực hiện; kết quả đúng theo điều kiện tìm kiếm và sắp xếp.
Các bước thực hiện	1. Người dùng truy cập trang Active History. 2. Frontend gửi yêu cầu GET /api/devices/history lấy dữ liệu lịch sử đến Backend. 3. Backend tiếp nhận yêu cầu và truy vấn dữ liệu lịch sử từ Database. 4. Database trả dữ liệu lịch sử cho Backend. 5. Backend trả danh sách lịch sử về Frontend. 6. Frontend hiển thị dữ liệu dưới dạng bảng. 7. Người dùng có thể nhập từ khóa tìm kiếm hoặc chọn cách sắp xếp. 8. Frontend gửi điều kiện tìm kiếm và sắp xếp đến Backend. 9. Backend xử lý yêu cầu và truy vấn dữ liệu phù hợp từ

	Database. 10. Backend trả dữ liệu đã được tìm kiếm và sắp xếp về Frontend. 11. Frontend cập nhật bảng lịch sử.

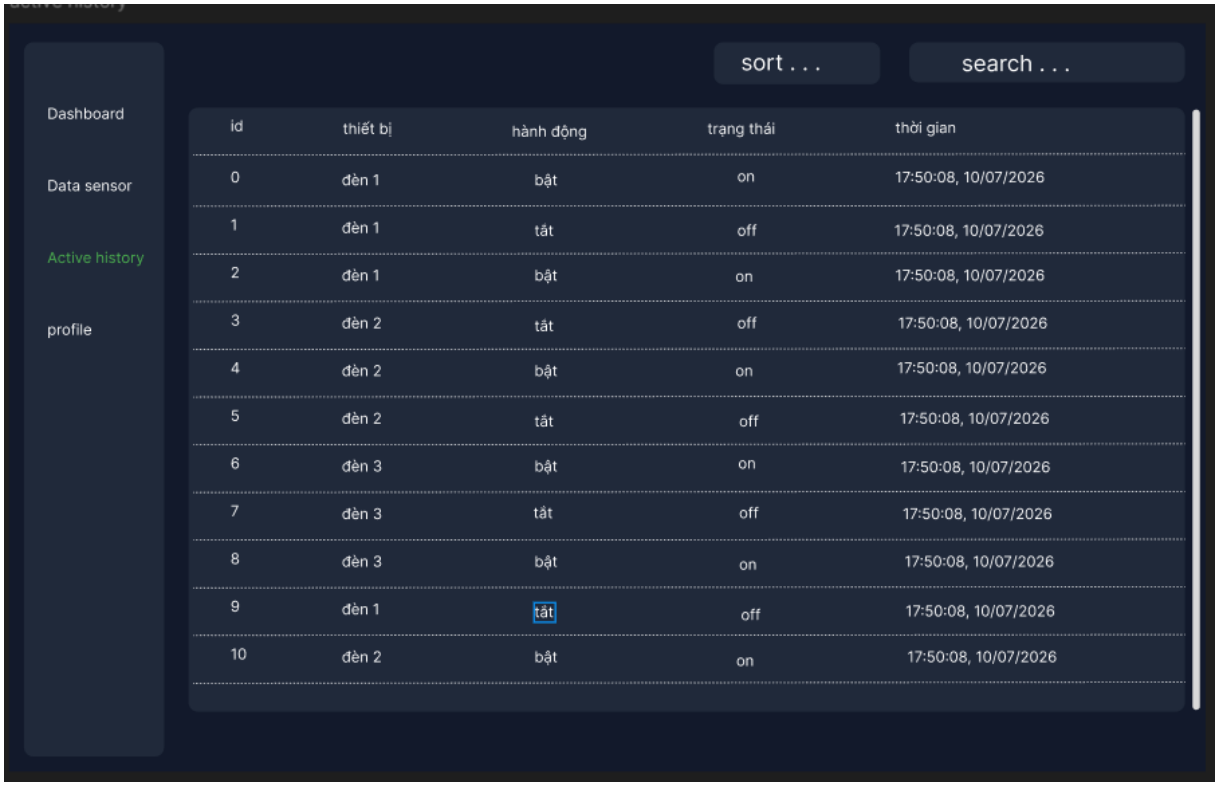
b. Biểu đồ tuần tự



Chi tiết luồng tương tác:

- **Bước 1 & 2:** Người dùng truy cập trang Active History từ thanh điều hướng. Frontend gửi yêu cầu lấy danh sách lịch sử thao tác thiết bị lên Backend qua API GET /api/devices/history.
- **Bước 3 & 4:** Backend nhận yêu cầu và truy vấn dữ liệu lịch sử từ bảng active_history trong Cơ sở dữ liệu (Database). Database trích xuất và trả các bản ghi nhật ký về cho Backend.
- **Bước 5 & 6:** Backend chuẩn hóa dữ liệu thành chuỗi JSON và trả về Frontend. Frontend tiếp nhận và hiển thị lên bảng cuộn dọc: ánh xạ trường action thành ngôn ngữ hiển thị trực quan (bật/tắt), đồng thời tại cột trạng thái, hệ thống hiển thị trạng thái thực tế cuối cùng của thiết bị nhận được từ phản hồi (on/off tương ứng khi lệnh đạt SUCCESS)..
- **Bước 7 & 8:** Người dùng nhập từ khóa tìm kiếm hoặc chọn tiêu chí sắp xếp trên thanh điều khiển. Frontend gửi các tham số điều kiện đó lên Backend.
- **Bước 9, 10 & 11:** Backend xây dựng câu truy vấn có điều kiện phù hợp gửi xuống Database. Database trả kết quả dữ liệu đã lọc/sắp xếp về Backend; Backend trả kết quả đã xử lý về Frontend để cập nhật lại bảng danh sách cho người dùng.

c. Giao diện



d. Mô tả API: Tra cứu lịch sử hoạt động

API: Lấy danh sách lịch sử hoạt động thiết bị

- **Chức năng:** Trích xuất toàn bộ nhật ký thao tác điều khiển thiết bị kèm trạng thái thực tế và thời gian thực hiện.
- **Method (Phương thức):** GET
- **Endpoint (Đường dẫn): /api/devices/history**
- **Query Parameters:** limit=100&search=...&sort=asc
- **Output trả về (Thành công - Status 200 OK):**

JSON


```
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "device_name": "đèn 1",
      "action": "ON",
      "status": "SUCCESS",
      "time": "17:50:08, 10/07/2026"
    },
    {
      "id": 2,
      "device_name": "đèn 1",
```

```
    "action": "OFF",
    "status": "FAILED",
    "time": "17:55:12, 10/07/2026"
   }
 ]
}
```


- **Output trả về (Thất bại - Status 500 Internal Server Error):**

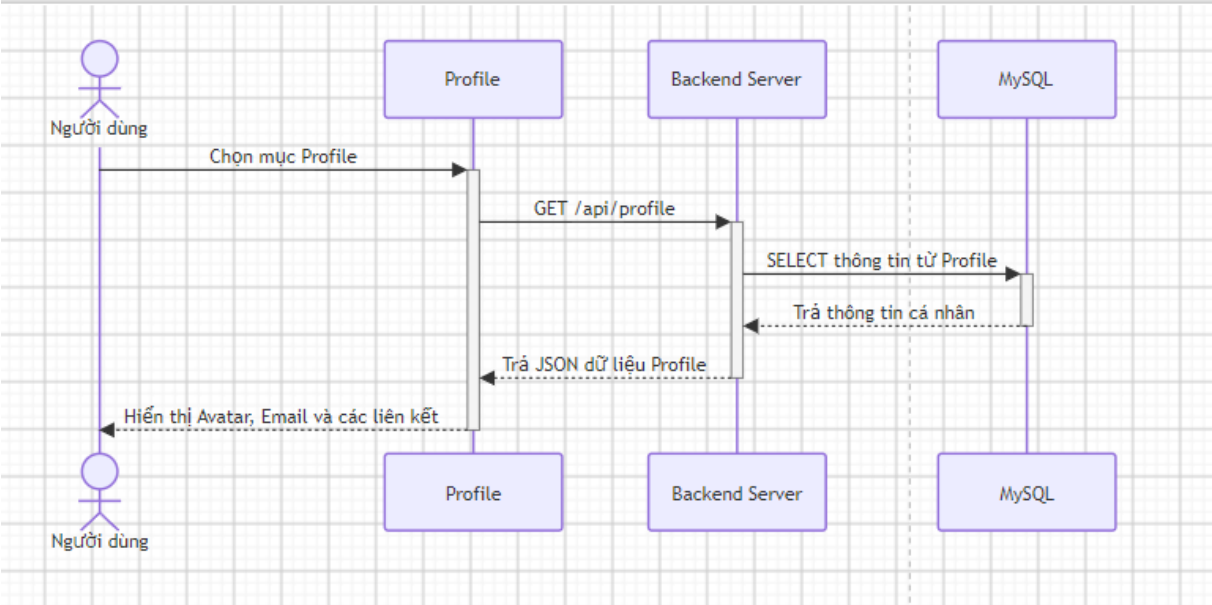

```
JSON
{
 "status": "error",
 "message": "Không thể kết nối cơ sở dữ liệu để lấy nhật ký hoạt động!"
}
```


3.2.5. Use Case 5 - Xem thông tin profile

a. Bảng Use case

Thuộc tính	Chi tiết
Tên Use case	Xem thông tin cá nhân
Mô tả	Trang hiển thị các thông tin cá nhân của người phát triển dự án và cung cấp các đường dẫn liên kết đến Github và Figma.
Tiền điều kiện	Không có.
Điều kiện thành công	Giao diện hiển thị đúng thông tin cá nhân và các đường link điều hướng hoạt động bình thường.
Các bước thực hiện	1. Người dùng chọn mục Profile trên thanh menu. / 2. Giao diện Web gọi API GET /api/profile xuống Backend. / 3. Backend truy vấn bảng Profile lấy dữ liệu cấu hình. / 4. Backend trả dữ liệu JSON. / 5. Web hiển thị Avatar, Email và các liên kết Github, Figma ra màn hình.

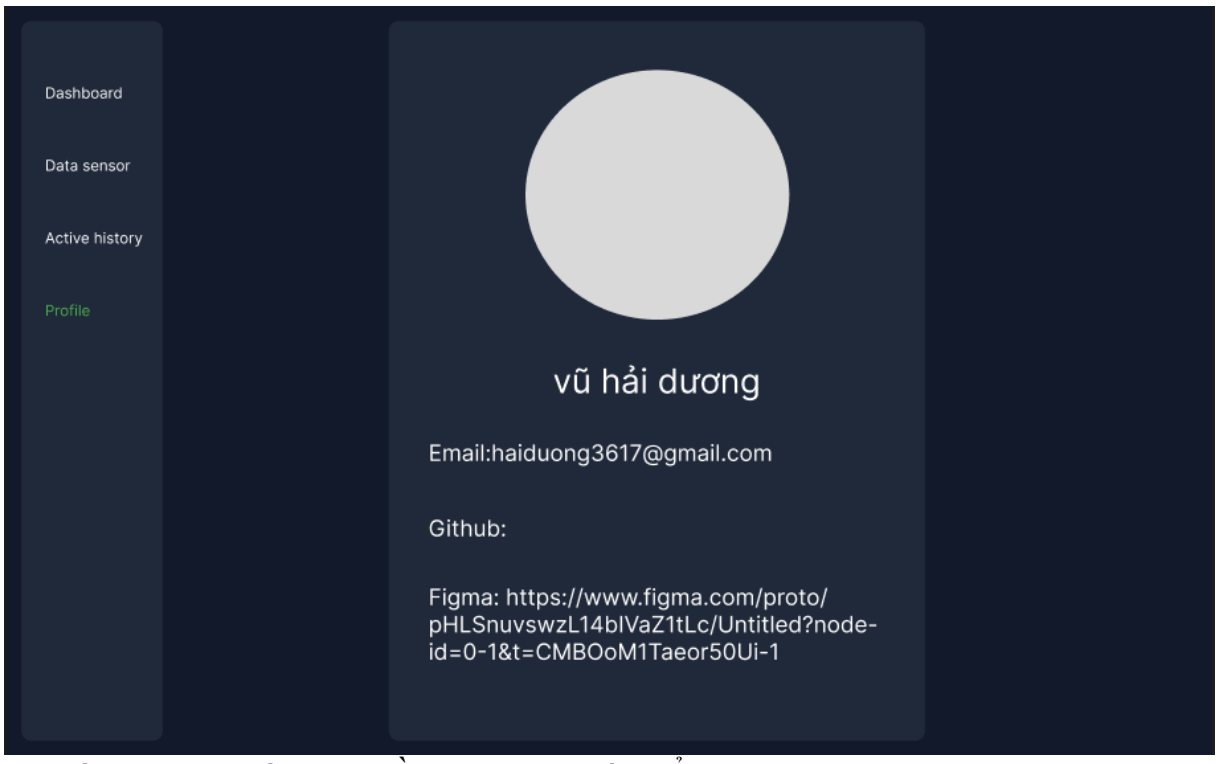
b. Biểu đồ tuần tự



Chi tiết luồng tương tác:

- **Bước 1 & 2:** Người dùng nhấn chọn mục Profile trên menu giao diện. Frontend gửi request GET /api/profile lên Backend Server để yêu cầu lấy thông tin cá nhân của người phát triển.
- **Bước 3 & 4:** Backend Server gửi câu truy vấn SELECT dữ liệu cấu hình từ bảng profile trong Cơ sở dữ liệu (MySQL). MySQL xử lý và trả về thông tin bản ghi hồ sơ tương ứng.
- **Bước 5:** Backend Server đóng gói thông tin thành JSON chứa tên, email, link Github và link Figma rồi phản hồi về cho Frontend.
- **Bước 6:** Frontend nhận dữ liệu, hiển thị Avatar, Email cùng các đường dẫn liên kết điều hướng trực tiếp ra màn hình cho người dùng.

c. Giao diện



d. Mô tả API: Thông tin hồ sơ người phát triển

API: Lấy thông tin Profile

- **Chức năng:** Cung cấp thông tin cá nhân và các liên kết dự án (Github, Figma) của người phát triển hệ thống.
- **Method (Phương thức):** GET
- **Endpoint (Đường dẫn):** /api/profile
- **Output trả về (Thành công - Status 200 OK):**

JSON

```
{
  "status": "success",
  "data": {
    "name": "vũ hải dương",
    "email": "haiduong3617@gmail.com",
    "gitlink": "https://github.com/...",
    "figmalink": "https://www.figma.com/..."
  }
}
```

- **Output trả về (Thất bại - Status 404 Not Found):**

JSON

```
{
  "status": "error",
  "message": "Không tìm thấy thông tin hồ sơ của người dùng trong hệ thống!"
}
```

Chương 4 Kết luận

Báo cáo đặc tả yêu cầu phần mềm (SRS) đã trình bày đầy đủ kiến trúc tổng quan, thiết kế cơ sở dữ liệu và chi tiết các ca sử dụng (Use Cases) của hệ thống IoT giám sát môi trường và điều khiển thiết bị.

Hệ thống đáp ứng tốt các mục tiêu đặt ra:

- Thu thập và hiển thị trực quan các thông số môi trường (nhiệt độ, độ ẩm, ánh sáng) từ phần cứng ESP32 lên giao diện Web theo chu kỳ định kỳ.
- Cung cấp khả năng điều khiển bật/tắt thiết bị đèn linh hoạt thông qua giao thức MQTT với cơ chế phản hồi trạng thái hai pha đảm bảo tính toàn vẹn và nhất quán.
- Lưu trữ đầy đủ lịch sử dữ liệu cảm biến và nhật ký thao tác thiết bị, hỗ trợ người dùng tra cứu, tìm kiếm và sắp xếp thuận tiện.

Đây là cơ sở kỹ thuật hoàn chỉnh để tiến hành triển khai, kiểm thử và vận hành hệ thống trong thực tế.